OPERATING SYSTEMS ORANGE LEVEL ASSIGNMENT

Akshat Sharma
PES1UG24CS042
4A

Phase 1:

Make test_objects and ./test_objects is ran :

```
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ ./test_
-bash: ./test_: No such file or directory
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$
```

Phase 1.2 :

./pes init is ran to initialize an empty repo

```
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ ./pes init
Initialized empty PES repository in .pes/
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$
```

Phase 2.1 :

make test_tree is ran and ./test_tree works and passes tests.

```
make: *** [Makefile:14: tree.o] Error 1
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ make
make: 'pes' is up to date.
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ make test_tree
make: 'test_tree' is up to date.
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Phase 2.2 : .

/test_tree is ran and find.pes just shows directory listings

```
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ find .pes
.pes
.pes/objects
.pes/refs
.pes/refs/heads
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$
```

PHASE 3.1 :

Empty repo is initialized , and 2 files are added using git-like commands. Then status is checked (with staging and unstaged and untracked files detailed view)

```
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ ./pes init
Initialized empty PES repository in .pes/
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ ./pes add file1.txt file2.txt
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  pes.h
  untracked:  test_sequence.sh
  untracked:  pes_asan
  untracked:  Makefile
  untracked:  tree.c
  untracked:  commit.h
  untracked:  .gitignore
  untracked:  README.md
  untracked:  index.c
  untracked:  test_tree.c
  untracked:  pes.c
  untracked:  object.c
  untracked:  tree.h
  untracked:  commit.c
  untracked:  test_objects.c
  untracked:  index.h

akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ cat .pes/index
100644 ebf0f28f3e0b799ef8aa5cc9614366d92ee232896ac6337d30325c31ed8669e0 1776669252 6 file1.txt
100644 28f838ab75e7f7f020a093ce0beeb8a2d4aab8e7cef7ae3ed4c8c65c7beda6ba 1776669252 6 file2.txt
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$
```

Phase 3.2 :

Testing by adding 2 files again manually.

```
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ cd ~/pes-lab/os-u4-orange-problem
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ make pes
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
rmakshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ rm -rf .pes
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ cd ~/pes-lab/os-u4-orange-problem
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ ./pes init
Initialized empty PES repository in .pes/
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ printf "hello\n" > file1.txt
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ printf "world\n" > file2.txt
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ ./pes add file1.txt file2.txt
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  pes.h
  untracked:  test_sequence.sh
  untracked:  pes_asan
  untracked:  Makefile
  untracked:  tree.c
  untracked:  commit.h
  untracked:  .gitignore
  untracked:  README.md
  untracked:  index.c
  untracked:  test_tree.c
  untracked:  pes.c
  untracked:  object.c
  untracked:  tree.h
  untracked:  commit.c
  untracked:  test_objects.c
  untracked:  index.h

akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ |
```

Phase 3.3 :

Extra step to show index file content

```
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ cd ~/pes-lab/os-u4-orange-problem
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776669470 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776669472 6 file2.txt
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$
```

PHASE 4.1:

WORKING FLOW by initializing an empty repo - adding files - committing changes and printing log (similar to git log –oneline or git log)

```
rakshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ rm -rf .pes
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ cd ~/pes-lab/os-u4-orange-problem
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ ./pes init
Initialized empty PES repository in .pes/
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ printf "Hello\n" > hello.txt
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ ./pes add hello.txt
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ ./pes commit -m "Initial"
Committed: 5a874dcc2bfb... Initial
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ printf "World\n" >> hello.txt
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ ./pes add hello.txt
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ ./pes commit -m "Add_world"
Committed: 6d489a105ccc... Add_world
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ printf "Goodbye\n" > bye.txt
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ ./pes add bye.txt
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ ./pes commit -m "Add_farewell"
Committed: 43f9fbefd1cf... Add_farewell
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ ./pes log
commit 43f9fbefd1cfbade5828e3c6aab5ac6355758f6d699c75524045c356951274a6
Author: PES User <pes@localhost>
Date:   1776670017

    Add_farewell

commit 6d489a105ccc3a799601d5f03bb9c492e53c7d41492caf57284884dc7657567a
Author: PES User <pes@localhost>
Date:   1776670011

    Add_world

commit 5a874dcc2bfb19a23ec39d603f57bfcf386e7dcd44191a1241c7e7da9e72df9b
Author: PES User <pes@localhost>
Date:   1776670003

    Initial

akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ |
```

Phase 4.2 :

```
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ cd ~/pes-lab/os-u4-orange-problem
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/43/f9fbefd1cfbade5828e3c6aab5ac6355758f6d699c75524045c356951274a6
.pes/objects/5a/874dcc2bfb19a23ec39d603f57bfcf386e7dcd44191a1241c7e7da9e72df9b
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6d/489a105ccc3a799601d5f03bb9c492e53c7d41492caf57284884dc7657567a
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$
```

Phase 4.3:

Full test history

```
.pes/refs/heads/main
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ cd ~/pes-lab/os-u4-orange-problem
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ echo "refs/heads/main:"
refs/heads/main:
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ cat .pes/refs/heads/main
43f9fbefd1cfbade5828e3c6aab5ac6355758f6d699c75524045c356951274a6
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ echo "HEAD:"
HEAD:
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ cat .pes/HEAD
ref: refs/heads/main
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: edcc94a8c274... Initial commit

Log after first commit:
commit edcc94a8c274837044b22f2c778a4a06de0e538da4846c0605392619b6f493d1
Author: PES User <pes@localhost>
Date:   1776670122

    Initial commit


--- Second Commit ---
Committed: ffe56baafdf6... Update file.txt

--- Third Commit ---
Committed: 4974dc4e1ab8... Add farewell

--- Full History ---
```

```
--- Full History ---
commit 4974dc4e1ab893b64a74d60c834d2e90093385b2276e82f29471de2d8870c08b
Author: PES User <pes@localhost>
Date:   1776670122

    Add farewell

commit ffe56baafdf6c99b28e8e23da76e4454b4da9f98c904e9e4e415a7616d9f6e37
Author: PES User <pes@localhost>
Date:   1776670122

    Update file.txt

commit edcc94a8c274837044b22f2c778a4a06de0e538da4846c0605392619b6f493d1
Author: PES User <pes@localhost>
Date:   1776670122

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
4974dc4e1ab893b64a74d60c834d2e90093385b2276e82f29471de2d8870c08b

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/49/74dc4e1ab893b64a74d60c834d2e90093385b2276e82f29471de2d8870c08b
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/ed/cc94a8c274837044b22f2c778a4a06de0e538da4846c0605392619b6f493d1
.pes/objects/ff/e56baafdf6c99b28e8e23da76e4454b4da9f98c904e9e4e415a7616d9f6e37

=== All integration tests completed ===
akshat@AkshatDev:~/pes-lab/os-u4-orange-problem$ |
```

NOTE: In the below picture are the <u>commits</u> with <u>actual file changes</u>. I have done –alow-empty commits to show commits that explain the changes step by step later on … <u>Please</u> ignore those and check these initial phase commits with the <u>required</u> file changes :

Phase 4.1: implement commit_create (tree build, parent lookup, metadata stuffff, object write, HEAD update)
Akshatpy committed 1 minute ago

Third phase implemented : index.c load, save, add and status
Akshatpy committed 12 minutes ago

Second Phase : implemented tree_from_index
Akshatpy committed 22 minutes ago

First phase : implement object_write
Akshatpy committed 37 minutes ago

Changing to test commit
Akshatpy committed 41 minutes ago

PHASE 5 AND 6 ANSWERS:

(These are already committed per subquestion in Readme.MD in the repo)

```
**Q5.1:** A branch in Git is just a file in `.git/refs/heads/` containing
a commit hash. Creating a branch is creating a file. Given this, how would
you implement `pes checkout <branch>` — what files need to change in
`.pes/`, and what must happen to the working directory? What makes this
operation complex?

### Ans 5.1
To implement `pes checkout <branch>`, I would do three things:
1. Validate the branch exists in `.pes/refs/heads/<branch>`.
2. Update `.pes/HEAD` to point to `ref: refs/heads/<branch>`.
3. Read the commit hash from that branch, load its root tree, and rewrite
the working directory to match that snapshot (create/update/delete files
as needed).

The tricky part is not updating HEAD; it is safely changing the working
directory. You have to avoid deleting user changes, handle nested trees
correctly, and make sure index + working directory + HEAD all end up
consistent.

**Q5.2:** When switching branches, the working directory must be updated
to match the target branch's tree. If the user has uncommitted changes to
a tracked file, and that file differs between branches, checkout must
refuse. Describe how you would detect this "dirty working directory"
conflict using only the index and the object store.
```

Ans 5.2
I would detect conflicts by comparing three versions of each tracked path:
1. Version in current HEAD tree (baseline).
2. Version in target branch tree (what checkout wants to write).
3. Current working directory state (using index metadata/hash checks).

If a file is modified in working directory relative to index/HEAD, and that same file would also change when moving to target branch, that is a conflict and checkout should stop.

In short: if file is dirty locally and target branch wants a different content for that path, refuse checkout.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Ans 5.3
In detached HEAD, new commits are still created normally, but no branch name moves forward with them. So those commits can become hard to find later.

Recovery is simple if you know the commit hash: create a new branch pointing to it, like `git branch rescued <hash>` (or equivalent in PES). If you realize late, reflog-style history (if implemented) helps find that commit again.

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Ans 6.1
I would use a classic mark-and-sweep approach:
1. Start from all branch heads in `.pes/refs/heads/*` as roots.
2. Traverse reachable commits via parent links.

3. From each commit, mark its tree.
4. Recursively mark all subtree and blob objects referenced by trees.
5. After marking, scan `.pes/objects` and delete anything unmarked.

For fast lookup, use a hash set of object IDs (`reachable`).

For 100,000 commits and 50 branches, commit traversal is usually near 100,000 unique commits (branches share history). Total visited objects could be a few hundred thousand when trees and blobs are included, depending on project size and change frequency.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Ans 6.2
Running GC during commit creation is risky because commit is multi-step. Example race:
1. Commit process writes new blob/tree objects.
2. Before branch ref is updated, GC scans refs and does not see these new objects as reachable.
3. GC deletes them.
4. Commit then writes/updates commit ref that points to now-missing objects.

Git avoids this using safety mechanisms like temporary reachability protection, conservative pruning (age-based), and coordination/locking so in-progress objects are not collected immediately.